Région Hauts-de-France
Cofinancé par l'Union européenne
realconseil DIAGNOSTIC ACCOMPAGNEMENT FORMATION

Manipulation de JSON en JavaScript

1. Qu'est-ce que le JSON ?

Le **JSON** (JavaScript Object Notation) est un **format texte** utilisé pour échanger des données entre un serveur et un client.

Il ressemble beaucoup aux objets JavaScript, mais il reste **une chaîne de caractères** tant qu'on ne l'a pas converti.

Exemple de JSON représentant un humain et un animal :

```
{
  "type": "Humain",
  "nom": "Manu",
  "amis": [
    {
      "type": "Chat",
      "nom": "Minou"
    },
    {
      "type": "Pigeon",
      "nom": "Piou"
    }
  ]
}
```

Un JSON peut contenir :

- des objets (`{ ... }`)
- des tableaux (`[ ... ]`)
- des nombres, des chaînes, des booléens, `null`

2. JSON ↔ Objets JavaScript

En JavaScript, on utilise deux fonctions globales très importantes :

- `JSON.parse()` pour **convertir une chaîne JSON** en **objet JavaScript**
- `JSON.stringify()` pour **convertir un objet JavaScript** en **chaîne JSON**

2.1. Convertir une chaîne JSON en objet JavaScript

```
const jsonTexte = `
{
  "type": "Humain",
  "nom": "Manu",
  "amis": [
    { "type": "Chat", "nom": "Minou" },
    { "type": "Pigeon", "nom": "Piou" }
  ]
}
`;

// Conversion en objet JS
const manuData = JSON.parse(jsonTexte);

console.log(manuData.nom);           // "Manu"
console.log(manuData.type);          // "Humain"
console.log(manuData.amis[0].nom);   // "Minou"
console.log(manuData.amis[1].type);  // "Pigeon"
```

À partir de là, `manuData` est un **objet classique**.
On peut l'utiliser comme n'importe quel objet que l'on a créé à la main.

2.2. Convertir un objet JavaScript en JSON

```
const manuObjet = {
  type: "Humain",
  nom: "Manu",
  espece: "Homo sapiens",
  animaux: [
    { type: "Chat", nom: "Rookie" },
    { type: "Pigeon", nom: "Piou" }
  ]
};

const jsonManu = JSON.stringify(manuObjet);

console.log(jsonManu);
// Résultat: une longue chaîne texte en JSON
```

Ce texte JSON est ce qui est **envoyé ou reçu** par une API REST.

Résumé important:

- **JSON.parse** : je pars d'un texte JSON, j'obtiens un objet JS
- **JSON.stringify** : je pars d'un objet JS, j'obtiens un texte JSON

3. Manipuler les données JSON comme des tableaux d'objets

Les API REST renvoient très souvent un **tableau d'objets JSON**.

Exemple : une liste d'êtres (humains, animaux, etc.) reçue depuis une API :

```
[
  { "nom": "Manu", "type": "Humain" },
  { "nom": "Clara", "type": "Humain" },
  { "nom": "Rookie", "type": "Chat" },
  { "nom": "Piou", "type": "Pigeon" },
  { "nom": "Croco", "type": "Crocodile" }
]
```

En JavaScript, après un `JSON.parse`, on obtiendra un **tableau** :

```
const jsonEtres = `
[
  { "nom": "Manu", "type": "Humain" },
  { "nom": "Clara", "type": "Humain" },
  { "nom": "Rookie", "type": "Chat" },
  { "nom": "Piou", "type": "Pigeon" },
  { "nom": "Croco", "type": "Crocodile" }
]
`;

const etres = JSON.parse(jsonEtres);

// Parcours simple
etres.forEach((etre) => {
  console.log(`${etre.nom} est un ${etre.type}`);
});
```

On peut ensuite utiliser toutes les méthodes de tableaux vues en cours :

3.1. Filtrer les humains seulement

```
const humains = etres.filter((etre) => etre.type === "Humain");
console.log(humains);
```

3.2. Récupérer les noms de tous les êtres

```
const noms = etres.map((etre) => etre.nom);
console.log(noms); // ["Manu", "Clara", "Rookie", "Piou", "Croco"]
```

3.3. Trouver un être par son nom

```
const recherche = "Piou";
const etreTrouve = etres.find((etre) => etre.nom === recherche);

if (etreTrouve) {
  console.log(`${etreTrouve.nom} est un ${etreTrouve.type}`);
} else {
  console.log(`Aucun être nommé ${recherche} trouvé.`);
}
```

4. Lien avec l'API REST (préparation)

Dans la partie suivante sur les **API REST**, on utilisera `fetch` pour appeler une URL.
Le schéma classique côté JavaScript sera :

```
fetch("https://exemple.com/api/etres")
  .then((response) => {
    if (!response.ok) {
      throw new Error("Erreur HTTP : " + response.status);
    }
    return response.json(); // <-- équivalent à JSON.parse sur le corps de
la réponse
  })
  .then((data) => {
    // Ici "data" est du JSON déjà transformé en objets JS / tableaux
    console.log("Données reçues :", data);
  })
  .catch((error) => {
    console.error("Problème lors de l'appel API :", error);
  });
```

Points clef:

- L'API renvoie du **JSON texte**
- Le navigateur le transforme en **objets JavaScript** via `response.json()`
- Ensuite, on manipule ces données avec :
  - l'accès aux propriétés (`data[0].nom`, `data[0].type`, etc.)
  - les méthodes de tableaux (`forEach`, `map`, `filter`, `find`, ...)
  - éventuellement nos **classes JS** (POO) si on veut transformer ces objets en vraies instances (`new Humain(...)`, `new Chat(...)`, etc.)

Exemple de transformation vers des instances de tes classes (en lien avec la POO JS) :

```javascript
// On suppose que Humain et Chat existent déjà (cf. cours POO)
function creerInstanceDepuisJSON(etre) {
  if (etre.type === "Humain") {
    return new Humain(etre.nom);
  }
  if (etre.type === "Chat") {
    return new Chat(etre.nom);
  }
  // Par défaut, un Animal générique
  return new Animal(etre.nom, etre.type);
}

const instances = etres.map(creerInstanceDepuisJSON);

instances.forEach((etre) => {
  etre.sePresenter(); // Polymorphisme : chaque type réagit à sa façon
});
```